Lucrarea 3 – Implementarea algoritmilor paraleli. Accesul concurent la resurse

Laborator Algoritmi Paraleli și Distribuiți – rezolvarea exercițiilor

EXERCIȚIUL 1

Am implementat și rulat codul din laborator (proiect Console Application, referință la `System.Drawing`):

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Drawing;
using System.IO;
using System.Diagnostics;

namespace L2
{
    class Program
    {
        static string sourceDirectory = @"C:\Studenti\SourceImages";
        static string finalDirectory = @"C:\Studenti\FinalImages";
        static string[] fileEntries;
        static int N = 4;

        static object _lock = new object();

        static void Main(string[] args)
        {
            fileEntries = Directory.GetFiles(sourceDirectory, "*.bmp").OrderBy(x => x).ToArray();
//Task A

            Thread[] childThreads = new Thread[N];
            for (int i = 0; i < N; i++)
            {
                childThreads[i] = new Thread(ThreadFunction);
                childThreads[i].Start(i);
            }
            for (int i = 0; i < N; i++)
            {
                childThreads[i].Join();
            }

            Console.ReadKey();
        }

        public static void ThreadFunction(object parameter)
        {
            int tid = (int)parameter; //thread ID
            bool exista = true;
            string fileName = string.Empty;
            while (exista)
            {
                exista = false;
                lock (_lock) //enter critical zone
                {
                    for (int i = 0; i < fileEntries.Length && !exista; i++)
                    {
                        if (!string.IsNullOrEmpty(fileEntries[i]))
                        {
                            exista = true;
                            fileName = fileEntries[i];
                            fileEntries[i] = string.Empty; //mark file as processed (by convention
an empty file == file is already taken)
                            Console.WriteLine("Thread ID {0} take image {1} ", tid, i);
                        }
```

```
                    }
                }
                if (exista)
                {
                    using (Bitmap image = new Bitmap(fileName)) //Task B
                    {
                        for (int x = 0; x < image.Width; x++)  //Task C
                        {
                            for (int y = 0; y < image.Height; y++)
                            {
                                Color pixelColor = image.GetPixel(x, y);
                                int gray = (pixelColor.R + pixelColor.G + pixelColor.B) / 3;
                                Color newColor = Color.FromArgb(gray, gray, gray);
                                image.SetPixel(x, y, newColor);
                            }
                        }
                        image.Save(Path.Combine(finalDirectory, Path.GetFileName(fileName)));
//Task D
                    }
                }
            }
        }
    }
}
```

Am testat cu 16 imagini de aceeași dimensiune (800x800 px) și cu 16 imagini de dimensiuni diferite (între 100x100 și 2000x2000 px), pentru N = 1, 2 și 4.

Exemplu de rulare, imagini de dimensiuni diferite, N = 4:

```
Thread ID 0 take image 0                        Thread ID 3 take image 8
Thread ID 2 take image 1                        Thread ID 3 take image 9
Thread ID 1 take image 2                        Thread ID 1 take image 10
Thread ID 3 take image 3                        Thread ID 3 take image 11
Thread ID 2 take image 4                        Thread ID 3 take image 12
Thread ID 3 take image 5                        Thread ID 3 take image 13
Thread ID 3 take image 6                        Thread ID 2 take image 14
Thread ID 3 take image 7                        Thread ID 2 take image 15
```

Observații:

- **N = 1:** un singur thread preia toate imaginile, pe rând, în ordinea din listă (0, 1, 2, ..., 15).
- **Imagini de aceeași dimensiune:** imaginile sunt preluate tot în ordinea din listă, iar threadurile le iau aproximativ pe rând. Fiecare thread procesează același număr de imagini (N = 2: 8 + 8, N = 4: câte 4).
- **Imagini de dimensiuni diferite:** ordinea de preluare rămâne cea din listă, dar threadurile nu mai procesează același număr de imagini (la N = 4: 9, 4, 2 și 1 imagini). Threadul care a primit o imagine mare rămâne ocupat mai mult, iar threadurile libere preiau imaginile următoare. Astfel taskurile se împart dinamic, după cât de repede termină fiecare thread.

EXERCIȚIUL 2

În varianta de la exercițiul 1, la fiecare preluare threadul parcurge lista de la început în zona critică, iar `Console.WriteLine` (o operație lentă) se execută tot în zona critică. Propunere: păstrăm un index `next` al următoarei imagini nepreluate. În zona critică doar citim și incrementăm indexul, iar restul operațiilor se fac în afara ei:

```
        static int next = 0;
        ...

        public static void ThreadFunction(object parameter)
        {
```

```
            int tid = (int)parameter; //thread ID
            bool exista = true;
            string fileName = string.Empty;
            while (exista)
            {
                int i;
                lock (_lock) //enter critical zone
                {
                    i = next;
                    next++;
                } //exit critical zone

                exista = i < fileEntries.Length;
                if (exista)
                {
                    fileName = fileEntries[i];
                    Console.WriteLine("Thread ID {0} take image {1} ", tid, i);

                    using (Bitmap image = new Bitmap(fileName)) //Task B
                    {
                        ... //Task C si Task D neschimbate
                    }
                }
            }
        }
```

Zona critică are acum doar două instrucțiuni, de durată constantă, indiferent de numărul de imagini, așa că threadurile stau mult mai puțin blocate în așteptarea lock-ului.

EXERCIȚIUL 3

Un lock pentru fiecare task. Vectorul `_locks` are aceeași lungime ca lista de imagini, iar threadul blochează doar elementul pe care îl verifică:

```
        //static object _lock = new object();
        static object[] _locks;

        static void Main(string[] args)
        {
            fileEntries = Directory.GetFiles(sourceDirectory, "*.bmp").OrderBy(x => x).ToArray();
//Task A

            _locks = new object[fileEntries.Length];
            for (int i = 0; i < _locks.Length; i++)
                _locks[i] = new object();

            ... //lansarea si asteptarea celor N threaduri, ca la exercitiul 1
        }

        public static void ThreadFunction(object parameter)
        {
            int tid = (int)parameter; //thread ID
            bool exista = true;
            string fileName = string.Empty;
            while (exista)
            {
                exista = false;

                for (int i = 0; i < fileEntries.Length && !exista; i++)
                {
                    lock (_locks[i]) //enter critical zone at task level
                    {
                        if (!string.IsNullOrEmpty(fileEntries[i]))
                        {
                            exista = true;
                            fileName = fileEntries[i];
                            fileEntries[i] = string.Empty; //mark file as processed (by convention
an empty file == file is already taken)
                            Console.WriteLine("Thread ID {0} take image {1} ", tid, i);
```

```
                    }
                }
            }

            if (exista)
            {
                ... //Task B, C, D neschimbate
            }
        }
    }
```

Accesul la listă nu mai este serializat: două threaduri pot verifica în același timp elemente diferite din listă și se blochează doar când încearcă să ia aceeași imagine.

EXERCIȚIUL 4

Două liste de taskuri, fiecare cu jumătate din fișierele din folder, și două variabile de tip lock. Threadurile cu ID par lucrează pe lista 1, iar cele cu ID impar pe lista 2:

```
    static string[] fileEntries1;
    static string[] fileEntries2;
    static int N = 4;

    static object _lock1 = new object();
    static object _lock2 = new object();

    static void Main(string[] args)
    {
        string[] fileEntries = Directory.GetFiles(sourceDirectory, "*.bmp").OrderBy(x =>
x).ToArray(); //Task A
        fileEntries1 = fileEntries.Take(fileEntries.Length / 2).ToArray();
        fileEntries2 = fileEntries.Skip(fileEntries.Length / 2).ToArray();

        ... //lansarea si asteptarea celor N threaduri, ca la exercitiul 1
    }

    public static void ThreadFunction(object parameter)
    {
        int tid = (int)parameter; //thread ID
        string[] fileEntries = (tid % 2 == 0) ? fileEntries1 : fileEntries2;
        object _lock = (tid % 2 == 0) ? _lock1 : _lock2;

        bool exista = true;
        string fileName = string.Empty;
        while (exista)
        {
            exista = false;
            lock (_lock) //enter critical zone
            {
                for (int i = 0; i < fileEntries.Length && !exista; i++)
                {
                    if (!string.IsNullOrEmpty(fileEntries[i]))
                    {
                        exista = true;
                        fileName = fileEntries[i];
                        fileEntries[i] = string.Empty; //mark file as processed (by convention
an empty file == file is already taken)
                        Console.WriteLine("Thread ID {0} (lista {1}) take image {2} ", tid, tid
% 2 + 1, i);
                    }
                }
            }
            if (exista)
            {
                ... //Task B, C, D neschimbate
            }
        }
    }
```

Fiecare listă are lock-ul ei, deci threadurile se blochează doar între ele pe aceeași listă (jumătate din threaduri pe fiecare lock), nu toate pe aceeași resursă. Soluția este una intermediară între un singur lock (exercițiul 1) și câte un lock pentru fiecare task (exercițiul 3).